

Detailed Report: Solar Production Forecasting with ARIMAX

1. Introduction

This project was developed for IE360 as a time series forecasting task. The objective is to forecast next-day hourly solar production for a solar power plant. The final output expected from the model is a list of 24 hourly forecast values.

Solar production forecasting is strongly affected by both time series structure and weather conditions. Production follows a daily daylight pattern, changes seasonally, and reacts directly to radiation and cloud cover. For that reason, the project uses an ARIMAX-style approach: a SARIMAX model captures time series dynamics while weather and calendar variables are supplied as exogenous regressors.

2. Data

2.1 Production Data

The dependent variable is 'sun_rt', representing hourly solar production. The data is downloaded from a Google Drive CSV file inside 'forecast_helper.py'.

The production table contains:

- 'dt': timestamp of the observation,
- 'sun_rt': observed solar production.

The timestamps are converted to the 'Europe/Istanbul' timezone before merging with weather data.

2.2 Weather Data

Weather data is downloaded from Open-Meteo. The project uses two coordinate points for the plant area:

- latitude '37.76967344769773', longitude '33.574324027291595',
- latitude '37.797667446516925', longitude '34.46742393001458'.

The variables used in the model are:

- 'temperature_2m',
- 'shortwave_radiation',
- 'cloudcover',
- 'relativehumidity_2m',
- 'weathercode'.

The code fetches both historical forecast data and current forecast data. Historical forecast data is used for past timestamps, while current forecast data supplies the future period needed for next-day prediction.

3. Data Preparation

The main script constructs an hourly timeline from '2022-01-01' to the final hour of the forecast day. Production data and weather data are merged into this hourly timeline by timestamp.

A 3-day lag variable is created:

```
sun_rt_lag_3days = sun_rt shifted by 72 hours
```

This lag is useful because in real forecasting settings the latest production data may not be immediately available. The lagged variable allows the model to use recent production behavior without relying on same-day observations.

4. Feature Engineering

The weather APIs return separate columns for each plant coordinate. The script combines the two locations into summary weather features:

- mean temperature,
- maximum temperature,
- mean shortwave radiation,
- mean cloud cover,
- mean relative humidity,
- mean weather code.

An additional derived feature is created:

```
effective_radiation = shortwave_radiation_mean * (1 - cloudcover_mean / 100)
```

This feature is a simple approximation of how much radiation remains after cloud cover is considered.

The hourly data is then aggregated into a daily table. The daily table contains:

- total daily production,
- maximum daily radiation,
- sum of the 3-day lagged production,
- daily weather averages,
- trend index,
- day-of-week category.

Several variables are transformed using 'log1p'. This reduces the impact of large values and keeps zero values valid:

```
log_x = log(1 + x)
```

Negative values are clipped to zero before applying the log transformation.

5. Modeling Approach

The project uses a SARIMAX model as an ARIMAX-style forecasting model. The model is configured with:

```
order = (1, 1, 1)
```

The dependent variable is the log-transformed total daily production. Exogenous variables include weather features, the 3-day lagged production feature, trend, and day-of-week dummy variables.

The model is trained using data that would realistically be available at forecast time. For a target day 't+1', the training data is limited to 't-2' and earlier. This avoids using production observations that may not yet be available.

The daily forecast is converted back to the original scale with:

```
forecast = exp(log_forecast) - 1
```

Negative forecasts are clipped to zero.

6. Hourly Forecast Distribution

The SARIMAX model forecasts daily total production. To produce the required hourly output, the daily forecast is distributed across 24 hours using historical hourly production profiles.

The hourly profile is built as follows:

1. Calculate each hour's share of its daily production total.
2. Group historical ratios by year, ISO week, and hour.
3. For the forecast date, use the matching year-week-hour profile when available.
4. If unavailable, fall back to the previous week.
5. If still unavailable, use the same ISO week averaged across all years.
6. If no profile exists, distribute the daily forecast equally across 24 hours.

This approach preserves the typical daily solar production shape: low values at night, increasing values after sunrise, peak values around midday, and decreasing values toward sunset.

7. Current Implementation

The cleaned implementation in 'forecast.py' is organized into focused functions:

- 'ForecastConfig': stores forecast date, start date, and timezone.
- 'fetch_weather_data()': downloads and combines weather data.
- 'build_hourly_dataset()': creates the merged hourly production-weather table.
- 'add_hourly_weather_features()': builds location-averaged weather features.
- 'build_daily_dataset()': aggregates hourly data into daily model rows.
- 'build_model_features()': creates numeric model inputs.
- 'forecast_daily_totals()': fits SARIMAX models and forecasts daily totals.
- 'build_hourly_profile()': calculates historical hourly production ratios.
- 'distribute_daily_forecasts()': converts daily forecasts into hourly values.
- 'run_forecast()': runs the end-to-end forecasting pipeline.

The only output of the script is the 24-value forecast list, which makes it easy to submit or consume programmatically.

8. Assumptions

The model relies on the following assumptions:

- The Google Drive production CSV is reachable and has the expected schema.
- Open-Meteo APIs are reachable at runtime.
- Weather forecasts are available for the forecast day.
- Historical hourly production patterns are useful for distributing daily totals.
- The 3-day lag is a realistic proxy for latest available production data.

- A fixed SARIMAX '(1, 1, 1)' structure is sufficient for this project version.

9. Limitations

The current implementation is functional but still has limitations:

- There is no committed offline dataset, so reproducibility depends on external services.
- The model does not automatically tune SARIMAX parameters.
- The script does not currently print evaluation metrics.
- The hourly distribution method is heuristic and separate from the SARIMAX model.
- API response changes or unavailable data can break the pipeline.
- 'forecast_helper.py' still has duplicated code for weather API handling.

10. Recommended Next Steps

Several improvements would make the project stronger:

- Add 'requirements.txt' to make setup reproducible.
- Add a local sample dataset for testing without internet access.
- Add a backtesting module that evaluates recent forecast performance.
- Report WMAPE, MAE, and RMSE over a rolling validation window.
- Compare SARIMAX against baseline models.
- Refactor 'forecast_helper.py' to share API request logic.
- Add command-line arguments for forecast date and output path.
- Save forecasts to CSV in addition to printing the list.

11. Conclusion

The project implements a practical solar production forecasting pipeline using weather data and time series modeling. The cleaned version separates data loading, feature engineering, daily forecasting, and hourly distribution into readable functions. This makes the project easier to understand, maintain, and extend for future IE360 reporting or experimentation.